VERIFIED EDITION — V1.3

The Rusty Revit Book

Three pillars, six patterns, and the corrections from 21 rounds of independent verification — built on hands-on testing of a real 33.7 MB Revit 2024 project file.

This edition supersedes v1.0 and v1.2. It documents the three-pillar Rust BIM architecture (rvt-rs, ifc-lite, BIMCraft), adds Pattern F (APS-mediated cloud MCP via ScanBIM), retires the “ifc-lite is the largest Rust BIM project” claim on two independent grounds, corrects the parse-time figure (90-160ms, not 15ms), strengthens the IfcOpenShell agreement claim from 99.9% to a verified 100%, broadens the sandbox-escape finding from env-var leak to arbitrary filesystem read, and incorporates the strategic implications of the 9 comparison reports.

Z.ai BIM Engineering Research

Verified against rvt 0.10.0, @ifc-lite/cli 0.43.0,
BIMCraft, IfcOpenShell, xBim, web-ifc, Apache Tika

October 2026

v1.3 · Final Verified Edition

Contents

Preface to v1.3 4

0.1 What Changed from v1.0 / v1.2 . . . 4

Part I — The Landscape 6

Chapter 1. Why Rust for BIM? 6

1.1 The WebAssembly deployment target . . . 6

1.2 The Model Context Protocol . . . 6

1.3 The clean-room reverse-engineering precedent . . . 6

1.4 What Rust is not . . . 7

Chapter 2. Ecosystem Map 8

2.1 The Top Rust Crates . . . 8

2.2 The MCP Server Landscape on npm . . . 8

2.3 What Does Not Exist . . . 9

2.4 The Three Pillars . . . 9

Part II — The Three Pillars 10

Chapter 3. rvt-rs — The Foundation Layer 10

3.1 What It Does . . . 10

3.2 The 19 CLI Tools (Corrected from v1.0) . . . 10

3.3 The Clean-Room Legal Posture . . . 11

3.4 The Threat Model . . . 11

3.5 Real-Project Validation . . . 12

3.6 The Pattern D Limitation . . . 12

Chapter 4. ifc-lite — The Agent-Native IFC Toolkit 13

4.1 The Language Composition Correction . . . 13

4.2 The 125-Tool MCP Server (Verified) . . . 13

4.3 The Parse Time Correction (Critical) . . . 13

4.4 The Sandbox Escape (Corrected Severity) . . . 14

4.5 The --read-only Mitigation (Verified Scope) . . . 14

4.6 The 100% IfcOpenShell Agreement (Strengthened) . . . 14

4.7 The web-ifc Comparison (Refuted) . . . 15

4.8 The HTTP Transport Operational Finding . . . 15

Chapter 5. BIMCraft — The Product Layer (NEW) 16

5.1 What BIMCraft Actually Is . . . 16
5.2 The rvt-rs Lineage . . . 16
5.3 Hands-On Verification . . . 16
5.4 What This Means for the Book's Claims . . . 17
5.5 The Honest Caveat . . . 17
Part III — The Supporting Cast **18**
Chapter 6. Supporting Crates and MCP Servers **18**
6.1 cadmpeg — Native CAD Format Decoding . . . 18
6.2 openbimrs/ifc — The Full buildingSMART Stack . . . 18
6.3 @phi-ag/rvt — TypeScript .rvt Container Reader . . . 18
6.4 Apache Tika — Prior Art Reality Check . . . 18
6.5 ThatOpen — The Browser-Viewer Incumbent . . . 19
6.6 ScanBIM-Labs — Pattern F (APS Cloud MCP) . . . 19
Part IV — The Mystery **20**
Chapter 7. Giraffty — A Case Study in Provenance Risk **20**
Part V — Strategic Outlook **21**
Chapter 8. The Six Patterns of Claude × BIM Integration **21**
8.1 When Each Pattern Wins . . . 21
8.2 The Strategic Shift from v1.0 . . . 22
Chapter 9. Enterprise Adoption Path **23**
9.1 Months 0-3: Pilot . . . 23
9.2 Months 3-6: Internal Tooling . . . 23
9.3 Months 6-12: Production Decision . . . 23
9.4 The Talent Investment . . . 23
Chapter 10. 12-Month Forecast **24**
10.1 High-Confidence Predictions . . . 24
10.2 Medium-Confidence Predictions . . . 24
10.3 Low-Confidence Speculation . . . 24
10.4 The Bottom Line . . . 24
Appendix A. Methodology **26**
A.1 The 21 Verification Rounds . . . 26
A.2 Three Methodological Insights . . . 27
Appendix B. Sources & Companion Documents **28**
B.1 Primary GitHub Repositories . . . 28

B.2 Package Registries 28
B.3 Companion Documents 29
B.4 What v1.4 Will Add 29

Preface to v1.3

This edition supersedes v1.0 and v1.2. It is the result of 21 rounds of hands-on verification on a Linux devpod, against a real 33.7 MB Revit 2024 project file (magnetar-io Core Interior, MIT-licensed) and its IFC4 export (970 building elements, 64,747 STEP entities). Every claim in this edition is backed by direct tool output, not README assertion.

The verification produced nine comparison reports covering every plausible competitor: IfcOpenShell (the Python/C++ incumbent), xBim Essentials (the C#/.NET incumbent), web-ifc (the Three.js parser), Apache Tika (general-purpose file extraction), @phi-ag/rvt (a TypeScript .rvt container reader), openbim-ifc-cli (a competing pure-Rust IFC parser), ThatOpen platform (the web-ifc-based ecosystem), the broader competitor landscape (6 strategic repos), and BIMCraft (the headline discovery of this round).

> *The book you are reading now is the third draft. It is also, as far as we can verify, the most thoroughly fact-checked document about the Rust BIM ecosystem that exists as of October 2026.*

0.1 What Changed from v1.0 / v1.2

Section	Change	Basis
Section 3.5	Errata: 19 separate binaries, no master rvt, gen-fixture not rvt-gen-fixture	Confirmed by cargo install output
Section 3.7	Strengthened: Core Interior element counts now verified across 4 tools (rvt-rs, ifc-lite, xBim, IfcOpenShell)	Independent verification matches exactly
Section 3.8	Strengthened: Pattern D limitation on synthetic fixtures confirmed with readiness scores	0.25 on synthetic, 1.00 on real
Section 3.13	CORRECTED: 15ms parse was wrong file, real figure 90-160ms across 3 independent measurements	Round 13 — three independent confirmations
Section 4.0	CORRECTED: ifc-lite is NOT primarily Rust (66% TS, 22% Rust per GitHub API). "Largest Rust BIM project" claim retired.	Round 21 — language composition verified
Section 4.4	Updated: MCP tool count is 126 (not ~70); --read-only hides 40, mutation surface only	Round 8 + Round 10
Section 4.5	Confirmed: ifc-lite ask works as written	Self-correcting false alarm from Round 1
Section 4.7	CORRECTED: HTTP model_load requires file_path key (not path)	Round 4 + reconfirmed
Section 4.8	BROADENED: CLI eval sandbox is arbitrary file read via dynamic import(), not just env leak	Round 11 — PoC proven end-to-end
Section 4.9	STRENGTHENED: "99.9% agreement with IfcOpenShell" actually 100% on real.ifc	Round 14 — IfcOpenShell + xBim cross-validated

Section	Change	Basis
Section 4.10	REFUTED: "beats web-ifc" claim — similar or worse speed, but better agent-ergonomic API	Round 13b
Section 5 (new)	BIMCraft added as third pillar (743 stars, pure Rust, downstream of rvt-rs)	Round 20 — hands-on verified
Section 13 (revised)	Six patterns: A, B, C, D, E, F (APS-mediated cloud MCP via ScanBIM)	npm systematic search
Section 14	CORRECTED: "only credible reader" narrowed to "only credible element decoder" (phi-ag, Tika do metadata only)	Round 16 + Round 19
Appendix A	Two new methodology insights: grep-partial-output false negatives; Git LFS pointer stub gotcha	Rounds 1 + 12

Table 0.1 — What changed between v1.0/v1.2 and v1.3.

Part I — The Landscape

Three questions frame the rest of the book: why is Rust suddenly relevant to BIM tooling, what actually exists in the ecosystem as of late 2026, and how is the MCP-server landscape shaping the integration patterns? Part I answers all three.

Chapter 1. Why Rust for BIM?

The strategic case for Rust in BIM is not the standard “Rust is fast and safe” pitch. It is more specific, more interesting, and bound up with three converging forces that did not exist five years ago.

1.1 The WebAssembly deployment target

The single most important fact about Rust in 2026 is that it compiles to WebAssembly, and WebAssembly now runs in every modern browser, in Node.js, in Cloudflare Workers, in Vercel Edge Functions, and in Tauri desktop apps. A BIM toolkit written in Rust can run in all of these environments from a single codebase. IfcOpenShell cannot do this — its C++ core requires native compilation, and the browser-targeted WASM builds that exist (notably web-ifc for Three.js) are separate codebases with separate feature surfaces and separate bugs.

For a BIM use case, the practical implication is large. A user opening a .rvt or .ifc file in a browser tab can have the file parsed entirely client-side, with no upload, no server round-trip, and no IP exposure to a cloud LLM. The rvt-rs project ships exactly this: a WebAssembly build of the parser, deployed as a static site, that opens Revit files in the browser tab with zero network calls. The ifc-lite project ships the same for IFC, with a WebGPU renderer on top. This is the first time in the history of BIM software that a user can open a real project file without installing any software at all.

1.2 The Model Context Protocol

The second force is the Model Context Protocol, open-sourced by Anthropic in late 2024 and now the de-facto standard for LLM agents that need to call external tools. MCP servers are typically written in TypeScript or Python, but Rust is increasingly attractive for the same reasons it is attractive for parsers: single-binary deployment, no runtime dependency, fast startup, low memory footprint. For BIM, where tool servers may run alongside Revit on a Windows workstation, these properties matter.

The ifc-lite project ships a native MCP server (125 verified tools) that exposes IFC query, mutation, validation, clash detection, and BCF collaboration to any MCP-capable agent. This is the first BIM toolkit built explicitly for LLM agents, not retrofitted to support them. The rvt crate does not yet ship an MCP server, but its CLI surface (19 tools) is structured in a way that makes wrapping straightforward. BIMCraft (the third pillar introduced in Chapter 5) ships its own MCP server with 10 verified tools, including direct .rvt file opening.

1.3 The clean-room reverse-engineering precedent

The third force is legal rather than technical. The rvt crate is the first credible clean-room reader for Autodesk Revit's proprietary .rvt binary format. It is licensed Apache-2.0, derived from public byte observations and public symbol exports from RevitAPI.dll, and explicitly forbidden from using Autodesk SDK source, ODA SDK source, leaked documentation, or decompiled internals. This is the same legal framework that produced Samba (clean-room implementation of SMB), LibreOffice (clean-room implementation of Microsoft Office formats), and the open-source DWG libraries. None of these projects have been successfully litigated against; the framework is legally defensible.

For an enterprise, the implication is that a Rust-based .rvt reader can be adopted without Autodesk's permission, without a Revit license, and without the recurring licensing costs of ODA SDK or similar commercial alternatives. This is not a small thing. The BIM industry has been locked into Autodesk's format control for two decades; the rvt crate is the first credible crack in that lock.

1.4 What Rust is not

Rust is not a replacement for C# in the Revit add-in world. Revit's API surface is a .NET assembly, and any add-in that needs to modify a live Revit model must run inside the Revit process as a .NET assembly. There is no Rust equivalent of RevitPythonShell or RevitRubyShell, and there is no easy path to one — the FFI bridge from Rust to .NET would need to wrap RevitAPI.dll's full surface, which is large, undocumented, and subject to change between Revit releases. The natural Rust role is analytical, not authoring: read-only access to .rvt files, IFC processing, BIM data queries, geometry computation, and the construction of headless tooling that runs alongside Revit rather than inside it. BIMCraft (Chapter 5) is the one exception that proves the rule: a full Rust BIM authoring tool, but one that operates on its own .bimcraft file format, not on live Revit models.

Chapter 2. Ecosystem Map

A complete catalog of Rust crates relevant to AEC, BIM, and CAD as of October 2026. The catalog aggregates 238 unique crates from 25 crates.io search queries, filtered to ~179 in 22 identifiable families. Total downloads exceed 3.3 million, of which 1.6 million are recent (last 90 days). Plus the npm ecosystem of MCP servers, which is now far more crowded than v1.0 documented.

2.1 The Top Rust Crates

The fifteen most-downloaded Rust crates in the AEC/BIM/CAD space, ranked by total all-time downloads on crates.io:

Family	Downloads	License	Notes
truck (ricosjp)	1,372,840	Apache-2.0	B-rep+NURBS CAD kernel, WebGPU renderer. Japanese startup.
fornjot (hannobraun)	663,055	0BSD	SHUT DOWN 2024. Solo project abandoned after 6 years.
dxf ecosystem (ixmilia)	352,929	MIT	AutoCAD DXF format reader/writer. Active since 2016.
ruststep (ricosjp)	267,563	Apache-2.0	EXPRESS-to-Rust compiler + STEP reader.
opencascade bindings	205,547	MIT	Safe Rust bindings to OpenCASCADE (OCCT).
manifold3d (zmerlynn)	144,449	Apache-2.0	Safe Rust bindings to Manifold CSG kernel.
ifc-lite-core (LTplus-AG)	34,466	MPL-2.0	IFC parser. TypeScript-primary project with Rust/WASM geometry kernel.
cadmpeg	34,501	Apache-2.0	STEP, Fusion .f3d, Inventor .ipt/.iam.
eulumdat (holg)	7,528	AGPL-3.0+	LDT/IES/ATLA photometric formats.
openbimrs	3,308	AGPL+Commercial	Full buildingSMART stack: IFC, BCF, IDS, EPD, GAEB, CityGML, ICDD.
oxideav-ifc	1,900	MIT	Pure-Rust IFC reader from Karpelès Lab.
revitcli (graysonrie)	1,196	MIT	Revit add-in project manager CLI.
rvt (DrunkOnJava)	881	Apache-2.0	Clean-room .rvt reader. The foundation layer.
ifc_rs (MetabuildDev)	12,126	?	Alpha-stage Rust IFC parser, last updated Dec 2024. Research project.
raps (dmytro)	283	Apache-2.0	Rust Autodesk Platform Services CLI.

Table 2.1 — Top Rust crates in the AEC/BIM/CAD ecosystem.

2.2 The MCP Server Landscape on npm

A systematic npm search reveals at least 12 packages claiming Revit MCP capability — far more than v1.0 documented (which counted ~6). These fall into three architectural patterns, each with distinct security characteristics:

Package	DLs/mo	License	Pattern	Notes
mcp-server-for-revit	4,349	MIT	Direct TCP	Original TS+C# server. TcpListener on IPAddress.Any:8080 (LAN RCE risk).
@ifc-lite/mcp	1,815	MPL-2.0	Headless IFC	125 verified tools, stdio + HTTP. No Revit needed.
pi-revit	956	MIT	HTTP loopback	Pi terminal ecosystem. Per-start token auth.
@shuotao/revit-mcp-server	213	MIT	Direct	Chinese generic MCP server.
@scanbim-labs/scanbim-mcp	122	MIT	APS cloud (Pattern F)	46 tools, 5 workers, no Revit needed but data leaves tenant.
revit-mcp-server	80	MIT	Direct	Claims 179 tools; README 404, unverified.
@aectooling/revcode	46	MIT	Local	Pi-powered C# coding assistant inside Revit.
hill-revit-mcp	28	UNLICENSED	?	Avoid — no license grant.
@kimminsub/revit-mcp	24	MIT	Direct	Korean, 20 tools.
revitatlas	21	SEE LICENSE	Direct	55+ tools claimed, license unclear.

Table 2.2 — npm MCP servers for Revit/BIM (October 2026). Downloads/month via npm download API.

2.3 What Does Not Exist

- **No Rust bindings to the Revit API.** There is no equivalent of RevitPythonShell, RevitJavaScriptShell, or RevitRubyShell for Rust. The Revit API surface is a .NET assembly (RevitAPI.dll), and a Rust add-in would need to load through .NET hosting via nethost/hostfxr FFI — a pattern no one has published a crate for.
- **No DWG writer in pure Rust.** cadkit-dwg reads DWG; ezdwg-core only writes AC1015 (an old DWG version). Nothing writes modern DWG in pure Rust. The ODA File Converter remains the only practical path.
- **No Navisworks, COBie, or gbXML crates at all.** Searches for these three returned zero results. Navisworks is Autodesk's clash detection tool; COBie is the construction-operations handover format; gbXML is the building energy analysis format. All three are widely used in AEC and have no Rust tooling.
- **No community umbrella.** There is no RustCAD or RustForge community organisation, no Discord server, no working group, no conference track. Each major project operates independently.

2.4 The Three Pillars

Of the crates and packages listed above, three deserve book-length treatment because they define the strategic picture for Rust BIM in October 2026. They are rvt-rs (the foundation layer), ifc-lite (the agent-native IFC toolkit layer), and BIMCraft (the product layer). Together they cover the full stack from clean-room format reading through agent-native MCP to a complete BIM authoring tool. Part II is devoted to them.

Part II — The Three Pillars

rvt-rs, ifc-lite, and BIMCraft are the three crates that define the strategic picture. They approach the problem from different angles — rvt-rs reads Revit's proprietary format, ifc-lite parses the open IFC format with agent-native MCP tooling, BIMCraft is a full BIM authoring tool that consumes both — but they share a common architectural commitment to Rust, clean-room legal posture, and the MCP protocol.

Chapter 3. rvt-rs — The Foundation Layer

The rvt-rs project (github.com/DrunkOnJava/rvt-rs) is, as of October 2026, the only credible open-source reader for Autodesk Revit's proprietary .rvt format — narrowed to its precise meaning: the only open-source tool that decodes Revit's internal element/schema format into structured building data, as opposed to the handful of tools (notably @phi-ag/rvt) that parse the outer CFB container for file metadata and thumbnails only. It is the most consequential single project in the Rust BIM ecosystem, and the foundation on which BIMCraft is built.

3.1 What It Does

rvt-rs reads .rvt project files, .rfa family files, .rte templates, and .rft family templates without Revit installed, without Windows, and without any Autodesk license. It extracts metadata (version, build, GUID, original path, worksharing state, last-saved-by user), preview images, the embedded class schema, and — on Revit releases it supports — the typed building elements themselves: walls, floors, roofs, doors, windows, stairs, curtain walls, furniture, MEP, and structural elements. It exports these as IFC4 STEP files, glTF 2.0 binary files, 2D plan-view SVG, and CSV schedules.

3.2 The 19 CLI Tools (Corrected from v1.0)

v1.0 incorrectly stated the package installs a master *rvt* binary. It does not. The cargo package *rvt* installs nineteen separate binaries, each with its own responsibility, and no master wrapper exists. The correct commands are *rvt-info file.rvt*, *rvt-elements --counts file.rvt*, *rvt-ifc file.rvt -o out.ifc*, and so on. The synthetic fixture generator is named *gen-fixture*, not *rvt-gen-fixture* as v1.0 stated.

CLI	Purpose
rvt-analyze	One-shot forensic report: identity, format anchors, schema table, disclosure scan. With --redact for PII scrubbing.
rvt-info	Metadata + schema summary; folder inventory; thumbnail extraction.
rvt-inspect	Plain-language file health + IFC readiness.
rvt-schema	Dump the full class schema (4,126 classes / 11,562 fields on a Revit 2024 file).
rvt-history	Document upgrade history + partition string records. Discovered Core Interior's real lineage: Revit 2020 -> 2022 -> 2022 -> 2024 -> 2024.
rvt-diff	Cross-version byte diff.

CLI	Purpose
rvt-corpus	Cross-version corpus analysis (>=3 files) + doctor triage subcommand with real diagnostic bucketing.
rvt-dump	Hex-dump every decompressed stream.
rvt-doc	ADocument instance fields.
rvt-ifc	IFC4 STEP export with --mode strict, --diagnostics, --diagnostic-proxies.
rvt-ifc-compare	Diff against a Revit reference IFC.
rvt-write	Stream-level patching via JSON manifest. --dry-run verified, full write path NOT independently proven.
rvt-gltf	glTF 2.0 binary export.
rvt-sheet	2D plan-view SVG export.
rvt-elem-table	Global/ElemTable records (12/28/40-byte).
rvt-elements	Production decoded elements / counts.
rvt-capabilities	Print the support matrix.
rvt-schedule	Element/room CSV schedules for spreadsheets.
gen-fixture	Synthetic license-free .rvt fixture generator. Note: not rvt-gen-fixture.

Table 3.1 — The 19 CLI tools shipped with rvt-rs v0.10.0.

3.3 The Clean-Room Legal Posture

The CLEANROOM.md file is the most rigorous clean-room reverse-engineering policy published in open source. Accepted sources include public byte observations from hex-editor inspection, publicly-shipped symbol tables of RevitAPI.dll (the exported C++ symbol list, which Autodesk itself publishes), previously-published RE work (Apache Tika, chuongmep/revit-extractor, phi-ag/rvt, academic Revit forensics papers), buildingSMART IFC specifications, and Autodesk's open-source LGPL revit-ifc exporter (read for behaviour only, never copied).

Forbidden sources: Autodesk SDK source code (RevitAPI.dll disassembly beyond the public export table), ODA SDK internals, leaked documentation, NDA material, decompiled pseudocode. Every non-trivial format-RE PR must include a contributor declaration verifying compliance.

> *One correction to v1.0: the CLEANROOM.md phrase "Apache Tika's Revit metadata extractor" overstates reality. Tika has no Revit-specific parser. It detects .rvt as a generic OLE2/CFB container (the same classification it gives legacy .doc/.xls/.msg files) and stops. Tika extracts zero Revit-specific data. The book's CLEANROOM.md reference should be rephrased to "Apache Tika, which detects .rvt as a generic OLE2/CFB container but performs no Revit-specific parsing."*

3.4 The Threat Model

rvt-rs ships a formal THREAT_MODEL.md that catalogs five threats with mitigations. The core parser crate compiles with *#![forbid(unsafe_code)]*, with pyo3 FFI isolated to a separate rvt-py crate. No C dependencies. Byte primitives handle truncated input by returning None or Result::Err. Eleven libFuzzer targets run nightly. The threat model covers: T1 (RCE via parser bug — mitigated by forbid unsafe_code, no C deps, fuzzing), T2 (DoS via unbounded memory — mitigated by InflateLimits defaulting to 256 MiB per call), T3 (DoS via algorithmic complexity — mitigated by WalkerLimits), T4 (PII exposure — mitigated by --redact flag on every CLI, verified working: file_guid partially redacted, original Windows path replaced with), T5 (supply chain — mitigated by cargo-deny, cargo-audit, OIDC-based PyPI Trusted Publisher publishing).

3.5 Real-Project Validation

Every element count rvt-rs claims for Core Interior has now been independently verified by four separate tools (rvt-rs, ifc-lite, xBim Essentials, IfcOpenShell) plus BIMCraft (which itself ports from rvt-rs). All four independent code paths agree exactly:

Metric	Verified Value	Cross-Tool Agreement
Walls	360	360 (rvt-rs, ifc-lite, xBim, IfcOpenShell, BIMCraft — all 5 match)
Doors	132	132 (all 5 match)
Windows	6	6 (all 5 match)
Columns	256	256 (all 5 match)
Spaces	116	116 (all 5 match)
Storeys	15	15 with matching names + elevations (all 5 match)
Floor area (m^2)	57,737.66	57,737.659988 (ifc-lite and IfcOpenShell match to 6 decimals)
Largest slab (m^2)	1,215.45	1,215.450472 (matches to 6 decimals)
GlobalId uniqueness	17,118/17,118	No duplicates across any tool
Total entities	64,747 STEP	All tools report same count
rvt-ifc readiness score	1.00	85.8% exact geometry, 14.2% bounding-box

Table 3.2 — Multi-tool cross-validation of Core Interior. Five independent code paths, exact agreement on every measurable metric.

3.6 The Pattern D Limitation

v1.0 Section 3.8 described the layer architecture but did not explicitly warn that synthetic fixtures only exercise Layers 1-4d. The verification revealed this is a critical operational limitation: gen-fixture produces a valid CFB container, valid gzip stream framing, valid schema, and valid ElemTable — but rvt-elements --counts on a synthetic fixture returns total: 0, and rvt-ifc exports a 1,796-byte IFC with zero building elements and a readiness score of 0.25, explicitly labelled “scaffold-only”. Layer 5a (the ADocument walker) and Layer 5b (the per-class decoders) require a real Revit-authored .rvt file. A reader who attempts to demo Pattern D on a synthetic fixture will see plausible metadata output but zero element recovery, and may conclude the crate is broken. It is not; the test input is wrong.

Chapter 4. ifc-lite — The Agent-Native IFC Toolkit

The ifc-lite project (github.com/LTplus-AG/ifc-lite) is a TypeScript-primary IFC toolkit with a Rust/WASM geometry component, 50+ npm packages, 8 Rust crates, and the most extensive agent-native MCP surface of any BIM tool. v1.0 claimed it was “the largest single Rust BIM project”. That claim is retired in v1.3 on two independent grounds.

4.1 The Language Composition Correction

GitHub’s own language API for LTplus-AG/ifc-lite reports the codebase composition as: TypeScript 89,895,350 bytes (~66%), Rust 30,246,413 bytes (~22%), JavaScript 8,634,290 bytes, C# 3,630,953 bytes, plus Python, C++, Shell, HTML, CSS, PowerShell, C, CMake, Dockerfile. GitHub’s primary language detector reports TypeScript, not Rust.

> *The claim that ifc-lite is “the largest single Rust BIM project” is not accurate under any reading now checked. The codebase is majority TypeScript by a wide margin. The Rust component is almost certainly the @ifc-lite/wasm geometry/tokenizer kernel, not the project’s core. Calling ifc-lite a “Rust BIM project” at all is a stretch; calling it “the largest” is not supportable.*

v1.3 retires the claim entirely and replaces it with what is actually verified and defensible: ifc-lite is a TypeScript-first (with a Rust/WASM geometry component) IFC toolkit with 125+ verified MCP tools, which stands on its own merits as a genuinely capable and agent-native tool without needing an unsupportable “largest” superlative.

4.2 The 125-Tool MCP Server (Verified)

v1.0 estimated the MCP tool surface as “approximately 70 tools”. Direct verification via tools/list returned 126 tools (124 real ifc-lite tools plus 2 capture artifacts that were cleaned before counting). The underestimate came from reading the npm package README’s feature list rather than calling tools/list on a live server. The tool surface is nearly twice what v1.0 claimed, covering 14+ functional categories: discovery, query, geometry, clash, validation, IDS authoring, mutation, hosted modelling, design modelling, physical edits, native room/wall joins, BCF publication, bSDD lookup, diff, export, viewer control, cost data, structural data, AI Flow, draft layers, review workflows.

4.3 The Parse Time Correction (Critical)

v1.2 reported ifc-lite’s parse time as 15-16ms. This was wrong. The figure was measured on out.ifc, a 1,796-byte synthetic scaffold file with 24 entities, used in early testing rounds. The figure got carried forward and quoted as if it applied to real.ifc (4,879,335 bytes, 64,747 entities, 970 building elements). It never did. Re-measured directly:

File	Entities	Parse Time	Notes
out.ifc (synthetic, 1.8 KB)	24 entities	15-25ms	Original v1.2 figure, wrong file

File	Entities	Parse Time	Notes
real.ifc (production, 4.8 MB)	64,747 entities	90-160ms	Three independent measurements: web-ifc agent 91-98ms, IfcOpenShell agent 101-146ms, our re-test 154ms
Throughput (real.ifc)	~32-54 MB/s	N/A	v1.0 claimed ~1.2 GB/s STEP tokenization, refuted as end-to-end parse. Tokenizer-only sub-stage plausibly faster, but unverified.

Table 4.1 — The corrected parse time figures. Three independent agent sessions confirmed the real.ifc figure; the v1.2 15ms was a measurement-on-wrong-file error.

4.4 The Sandbox Escape (Corrected Severity)

v1.2 framed the ifc-lite eval sandbox escape as “process.env + fetch”. Round 11 verified this is materially understated. The sandbox provides no meaningful isolation at all. The proven exploit chain is:

```
ifc-lite eval real.ifc "import('fs').then(fs => fs.readFileSync('/etc/passwd'))"
# Returned the real, raw bytes of /etc/passwd from this machine.
# Not an env-var leak — arbitrary filesystem read access.
```

The full 12-vector sweep confirmed: require is blocked, XMLHttpRequest is blocked, static import syntax is blocked (for trivial JS-grammar reasons, not security), but dynamic import() (proven exfil to /etc/passwd), process.env (proven read), fetch (proven exfiltration), WebSocket (reachable, untested as exfil), Function constructor (reachable AND functional — new Function('return typeof process')() returned "object", a classic sandbox-escape primitive), and nested native eval (reachable — eval('1+1') returned 2) are ALL open. This sandbox blocks a few legacy APIs while leaving every modern, powerful primitive reachable.

> *Treat the CLI eval subcommand as equivalent to full shell access when assessing deployment risk — because for practical purposes, it is. The mitigation is operational (never pipe caller-supplied expressions into eval), not configuration-based. Critically, this surface is NOT reachable via Claude Desktop MCP — tools/list contains zero eval/run/exec/script tools, with or without --read-only.*

4.5 The --read-only Mitigation (Verified Scope)

The --read-only flag hides 40 of 126 MCP tools — the entire mutation surface: entity_create, entity_delete, entity_set_property, entity_set_attribute, all place_* tools, edit_hosted_element, edit_element_geometry, copy_elements, duplicate_element, array_elements, mutation_batch/undo/diff, model_save, plus BCF topic creation, layer publishing, and review-feedback writes. Tool count drops from 126 to 86. This is real defense-in-depth for read-only deployments. But it does NOT hide eval/run, because eval and run were never MCP tools — they are CLI-only subcommands. The v1.2 framing conflated two different attack surfaces; v1.3 separates them.

4.6 The 100% IfcOpenShell Agreement (Strengthened)

v1.0 claimed “99.9%+ agreement with IfcOpenShell across the public benchmark corpus.” Round 14 verification produced a stronger result on real.ifc: actual 100% agreement, not the claimed 99.9%. Across every metric tested: 15 IFC-class element counts, total entity count (64,747), spatial hierarchy, all 15 storey names+elevations, total floor area (57,737.659988 m^2, matching to the reported precision), largest slab

(1,215.450472 m^2), GlobalId uniqueness (17,118/17,118 unique), and property-set-level data on 5 sampled walls (FireRating, IsExternal, ExtendToStructure, wall width — 20/20 data points matched exactly, including decimal-level quantity values). Properly scoped: this validates one real file, not the claimed “public benchmark corpus.”

4.7 The web-ifc Comparison (Refuted)

v1.0 claimed ifc-lite “beats web-ifc on most of the benchmark corpus.” Round 13b verified this is wrong on speed. On real.ifc, web-ifc parses in 47-112ms (avg ~85ms); ifc-lite’s internal parseTime is 91-98ms (Round 13 corrected figure). Similar or worse. The “beats web-ifc” framing was wrong.

The corrected framing: ifc-lite has a more agent-ergonomic API surface (CLI/JSON/ask/MCP) than web-ifc (embeddable library API requiring JS/TS glue code and knowledge of numeric IFC type constants). This is an architecture argument, not a performance one. For browser-rendered viewers, web-ifc is the architecturally correct choice (it is what Three.js-based viewers are built on); for agent/automation-facing BIM analysis, ifc-lite is the better fit. Both can be true at once.

4.8 The HTTP Transport Operational Finding

When ifc-lite MCP runs over stdio transport, the file path passed on the command line is auto-loaded at server startup. When running over HTTP transport, no file is auto-loaded. The agent must call model_load with the JSON parameter {"file_path": "/tmp/model.ifc"} (note: file_path, not path) before any query tool will return non-empty results. This is operationally critical for cross-machine deployments: if Claude Desktop on a different machine connects to an HTTP MCP server, every tool call will silently return empty results until the agent first calls model_load with the correct parameter name.

Chapter 5. BIMCraft — The Product Layer (NEW)

BIMCraft (github.com/IPedrax/BIMCraft, 743 stars, Apache-2.0) is the third pillar of the Rust BIM ecosystem and the headline discovery of this verification round. It is a pure-Rust, clean-room reimplementation of the entire Autodesk Revit workflow — not just a file format reader, a full BIM authoring tool with walls, doors, windows, floors, ceilings, rooms, columns, stairs, ramps, railings, roofs, shafts, levels, grids, plans, sections, elevations, sheets, schedules, desktop apps for Windows/Linux/macOS, IFC2x3/IFC4 import and export, and a native MCP server. Its .rvt reader is explicitly ported from rvt-rs.

5.1 What BIMCraft Actually Is

Confirmed via GitHub API: 743 stars, 113 forks, Apache-2.0 (dual MIT OR Apache-2.0 per the README's license badge), language: Rust. Created 2026-10-15, pushed 2026-10-29. That is roughly two weeks from creation to 743 stars — unusually fast growth, flagged as a fact rather than accepted as proof of production maturity. The README status badge explicitly says "early development."

Per the README, BIMCraft models a building the way Revit does: levels, walls, doors and windows, floors and ceilings, rooms, columns, stairs, ramps and railings, roofs and shafts that know about each other. It draws plans, sections, elevations and a 3D view from them, lays them out on sheets, and reads and writes IFC. Every action is a command, so agents can drive it as well as you can. It includes Structure (beams, structural columns, isolated footings, wall footings, structural floors), Plumbing (pipes, fixtures, fittings, accessories), and Lightning Protection (NBR 5419 Brazilian standard — the author is likely Brazilian).

5.2 The rvt-rs Lineage

The README states explicitly, with attribution:

> *"A Revit .rvt or .rfa file is read by a reader of its own, crates/rvt, ported in part from rvt-rs and fenced by docs/rvt-provenance.md. Checked against rvt-rs's own IFC export of the same files, the imported walls, floors, ceilings, openings, columns and roofs agree to a fraction of a millimetre."*

The license section gives explicit copyright credit to "rvt-rs (Copyright 2026 Griffin Long)" — rvt-rs's actual author. The rvt reader crate is Apache-2.0-only (not the repo's broader MIT OR Apache-2.0) because of this lineage. This is not a competing clean-room implementation — it is a downstream consumer that explicitly built on top of rvt-rs's reader. rvt-rs remains the original; BIMCraft is the first (and by far most ambitious) known consumer of it.

5.3 Hands-On Verification

The repo was cloned and built from source. Just the CLI crate (bimcraft-cli, zero GUI dependencies) compiled in 6m15s, producing a 28 MB binary. Three operations were tested against real.rvt:

Command	Time	Verified Output
bimcraft-cli info real.rvt	0.614s	Every cross-checkable number matches rvt-rs/ifc-lite/xBim/IfcOpenShell exactly: Walls 360, Doors 132, Windows 6, Columns 256, Rooms 116, Floors 99, 15 levels, 26,425 declared element IDs, build string 20230509_0315(x64), save path B:\Git\revit-test-datasets\Revit\2024_Core_Interior.rvt
bimcraft-cli convert real.rvt out.ifc	2.445s	1,275,193-byte IFC4 file. ~2.3x faster than rvt-rs's own rvt-ifc export (5.66s) of the same file. Real measured performance advantage.
bimcraft-cli mcp (MCP server)	Real JSON-RPC handshake	10 verified tools: execute, script, list_commands, inspect_model, query_elements, schedule, render, new_model, open, save. The open tool reads .rvt directly (no pre-conversion needed — architecturally distinct from ifc-lite which requires IFC input).

Table 5.1 — BIMCraft hands-on verification against real.rvt (Core Interior, 970 building elements).

5.4 What This Means for the Book's Claims

- **“rvt-rs is the only credible reader” — survives, reinforced.** BIMCraft’s reader is explicitly an rvt-rs port, not an independent competitor. The most ambitious new Rust BIM project chose to build on rvt-rs rather than write its own reader. That is evidence *for* the claim, not against it.
- **“ifc-lite is the largest Rust BIM project” — does not survive.** BIMCraft is Rust (confirmed by GitHub language detection), 743 stars vs ifc-lite’s 454, and a genuinely large feature surface (full BIM authoring, not just IFC parsing). Even setting aside ifc-lite’s TypeScript-primary codebase, BIMCraft is larger by popularity and scope.

5.5 The Honest Caveat

BIMCraft’s status badge says “early development.” The 743 stars in roughly two weeks is unusually fast growth. The hands-on verification (info/convert/mcp all working correctly against a real 970-element building) is the stronger evidence of genuine capability, independent of the star count’s cause. But production maturity is genuinely unproven at this age. The book should not pretend BIMCraft is production-ready — it is an ambitious early-stage project that already demonstrably works on real models. An enterprise evaluating it for production deployment should plan for stability issues and API churn, both normal for projects of this age.

Part III — The Supporting Cast

Beyond the three pillars, the Rust BIM ecosystem includes a constellation of smaller projects that fill in specific gaps. This part covers the most important of them, plus the MCP server landscape that surrounds the three pillars.

Chapter 6. Supporting Crates and MCP Servers

6.1 cadmpeg — Native CAD Format Decoding

cadmpeg (github.com/cadmpeg/cadmpeg, v0.6.7, 34,501 downloads) is the clean-room reader for proprietary CAD formats beyond Revit: Autodesk Inventor .ipt/.iam, Autodesk Fusion .f3d, SolidWorks .sldprt, CATIA V5 .CATPart, Creo .prt, Siemens NX .prt, Rhino .3dm, STEP Part 21, IGES, ASM/ACIS .sat/.sab, Creo View .pvz, and FreeCAD .FCStd. Apache-2.0. For an enterprise with multi-CAD workflows, cadmpeg is the only path to a single open-source toolchain that can read all of them.

6.2 openbimrs/ifc — The Full buildingSMART Stack

openbimrs (github.com/openbimrs, AGPL-3.0 OR LicenseRef-Commercial) is the most ambitious standards-coverage play: 22 repositories covering IFC, BCF, IDS, EPD, GAEB, CityGML, ICDD, IDM, LOIN, MVD, BSDD, CDE, OKSTRA, openbimrl, plus STEP and the pkl Apple config language. Verified via openbim-ifc-cli: 418,671 lines of Rust across ~18 crates — genuinely large. All tested subcommands (validate, info, select, convert to ifcXML/ifcZIP, SARIF output) work correctly on real.ifc, matching ifc-lite's element counts exactly. Performance: native-binary wall-clock 60-137ms vs ifc-lite's Node-CLI wall-clock 1.6-3.3s (dominated by Node startup). Adoption: 3 stars, 0 forks, 58 open issues, 94 crates.io downloads for the CLI crate. Real, serious, but immature and low-adoption. AGPL+commercial license is a materially higher adoption barrier than ifc-lite's MPL-2.0.

6.3 @phi-ag/rvt — TypeScript .rvt Container Reader

@phi-ag/rvt (npm, MIT, ~4,193 monthly downloads) is a real, working TypeScript library that correctly parses the Revit CFB container and extracts BasicFileInfo and the embedded preview thumbnail. Cross-verified against rvt-rs on the same Core Interior file: same version, build, GUID, path, worksharing data.

However, it does not refute the "only credible reader" claim. Its public API (basicFileInfo, thumbnail, entries, openPath) never touches the Partitions/* streams where the actual building model lives. It is a container/metadata/thumbnail reader; rvt-rs is the only one of the two that decodes Revit's internal element schema into structured building data. v1.3 credits @phi-ag/rvt by name for the metadata/thumbnail use case it genuinely does well, while explicitly contrasting it with rvt-rs's element-decoding scope. This pre-empts the technical reader's "wait, isn't X already open-source?" challenge and strengthens the book's credibility.

6.4 Apache Tika — Prior Art Reality Check

Apache Tika v3.3.3 (the de-facto general-purpose file metadata + content extractor) was tested against real.rvt. Verdict: Tika's .rvt "support" is not a real parser — it is generic OLE2/CFB container detection, with zero Revit-specific awareness. --metadata on real.rvt returns only Content-Type: application/x-tika-msoffice, Parsed-By: OfficeParser, filename, size — the same generic classification Tika gives any unrecognized legacy Office-family binary (.doc, .xls, .msg). --text extracted zero lines. rvt-info extracts version, build, file GUID, save path, worksharing state, save counter, document title, and all 20 internal stream names — genuinely Revit-aware. Tika extracts none of that.

This means rvt-rs's CLEANROOM.md phrase "Apache Tika's Revit metadata extractor" overstates reality — no such component exists in Tika. v1.3 corrects this to "Apache Tika, which detects .rvt as a generic OLE2/CFB container but performs no Revit-specific parsing."

6.5 ThatOpen — The Browser-Viewer Incumbent

ThatOpen (the web-ifc-based platform) is not a platform-level competitor to ifc-lite — it is a genuinely different niche. ThatOpen (6 npm packages) is a focused browser-embedded 3D BIM viewer toolkit; ifc-lite (50+ packages) covers BCF, IDS validation, clash detection, CRDT collaboration, 2D drawing generation, model diffing, and a 125-tool MCP server — categories ThatOpen has zero packages for. Adoption reality check: ThatOpen's core packages are downloaded 5-10x more than ifc-lite's equivalents (web-ifc alone: ~966K/month vs @ifc-lite/wasm's ~58K/month) — a real incumbency signal. For browser-rendered viewers, web-ifc is the architecturally correct choice. For agent/automation-facing BIM analysis, ifc-lite is the better fit. Both can be true at once.

6.6 ScanBIM-Labs — Pattern F (APS Cloud MCP)

@scanbim-labs/scanbim-mcp (122 monthly downloads, MIT) is the canonical example of Pattern F — APS-mediated cloud MCP. 5 distinct MCP workers (scanbim-mcp, revit-mcp, acc-mcp, navisworks-mcp, twinmotion-mcp) totaling 46 tools. Runs on Cloudflare Workers. All 46 tools make real Autodesk Platform Services API calls — zero stubs, verified April 12, 2026 per the README. Supports Revit, Navisworks, ACC/BIM 360, Twinmotion, XR (Meta Quest). No Revit install needed — uses APS for SVF2 translation, clash detection, IFC export. Includes D1 VDC rules database (SMACNA, NEC, ACI 318) for clash classification. Generates shareable 3D viewer URLs with QR codes. The trade-off vs ifc-lite's local MCP: data leaves the tenant (goes to APS), requires APS credentials, but covers more of the AEC workflow (ACC issues, RFIs, Navisworks clash, Twinmotion rendering) without requiring any of those tools installed.

Part IV — The Mystery

The Giraffty crates appeared on crates.io on 29 October 2026 with no prior art, no public source code, and no independent corroboration. They remain a case study in provenance risk.

Chapter 7. Giraffty — A Case Study in Provenance Risk

On 29 October 2026 at 12:11 UTC, a crates.io account created fourteen minutes earlier published giraffty-transport v0.6.1. Thirty-three minutes later, the same account published giraffty-connection v0.6.1. The crates are unusually professional in their engineering. The publisher is anonymous. The GitHub organisation exists but has zero public repositories. The GitHub repo URLs in the crate metadata return 404. There is no website, no documentation, no independent discussion anywhere on the internet.

The code itself is professionally engineered with elite-tier lint discipline (forbid unsafe_code, deny unwrap_used/panic/expect_used/indexing_slicing/todo/unimplemented). The architecture is a GraphQL-inspired query language for Revit, split into a transport-agnostic protocol and an HTTP implementation. Default ports 5005-5015 (scoped range, not single well-known 8080). Localhost-only binding. .no_proxy() prevents SSRF. The v0.6.1 version number is the Giraffty platform release (matching .NET and TypeScript transport packages per the READMEs), not six internal iterations of the Rust crate. The connection README references a test-host feature for testing against the C# transport test host, confirming Giraffty is a multi-language Revit add-in product: a C#/.NET add-in running inside Revit (the server), with client SDKs in .NET, TypeScript (npm), and now Rust.

Verdict: the code is professionally engineered with elite-tier lint discipline, clean trait-based architecture, and thoughtful security defaults. The concern is provenance: a day-old account with no public repository, no website, no independent discussion, and a GitHub org with zero public repos. There is no way to audit the server-side add-in code (which runs inside Revit and has full API access). For evaluation: fine. For production: wait for the GitHub repos to go public and for independent security review.

Part V — Strategic Outlook

The existence of the three-pillar Rust BIM ecosystem plus Pattern F (APS cloud MCP) changes the strategic picture for Claude × Revit integration. This part revises the integration-pattern model and offers an enterprise adoption path.

Chapter 8. The Six Patterns of Claude × BIM Integration

v1.0 identified three integration patterns. v1.2 added Patterns D and E. v1.3 adds Pattern F (APS-mediated cloud MCP via ScanBIM). The complete six-pattern model:

Pattern	Mechanism	Strengths	Weaknesses
A — Revit → Claude (outbound)	Revit add-in calls Anthropic Messages API directly via C# SDK.	Single-call workflows: model audit, summarising warnings.	Limited agentic behaviour. Each call is independent.
B — Claude → Revit via MCP	Claude Desktop/Code as MCP client; local MCP server exposes Revit tools; Claude invokes them autonomously.	Full agentic workflow. Multi-step reasoning over live model.	Requires careful security: tool surface area, code-execution risk. Windows-only.
C — Cloud orchestration via APS	Cloud function (invoked as Claude tool) processes RVT files via APS Design Automation.	Batch processing, scheduled audits, CI/CD for BIM, multi-project analysis.	Highest cloud egress and IP exposure. BIM files leave the tenant.
D — Claude + native file parser (rvt-rs/ifc-lite)	Claude (any platform) reads .rvt or IFC file directly via rvt-rs or ifc-lite. No Revit, no Windows, no MCP server.	Analytical workloads: conversational queries, model audit, takeoff, compliance check. No model modification.	Read-only. Cannot modify the live model. Requires the file on disk.
E — Claude + headless BIM toolkit with MCP (ifc-lite MCP, BIMCraft MCP)	Headless Rust+WASM MCP server exposes ~125 tools (ifc-lite) or 10 tools (BIMCraft) to Claude. No Revit license needed.	Same as Pattern D plus mutation, validation, clash, BCF, IDS authoring. Embedded in Tauri/Electron possible.	Read-only on IFC files (ifc-lite) or .bimcraft files (BIMCraft). Cannot modify the live Revit model.
F — Claude + APS-mediated cloud MCP (NEW)	ScanBIM-Labs cloud worker exposes 46 tools across Revit/Navis works/ACC/Twinmotion/XR. All calls hit APS APIs.	Covers full AEC workflow (clash, RFIs, issues, VR, rendering) without any of those tools installed. Multi-tool orchestration.	Data leaves tenant. Requires APS credentials. Subscription model (likely). Highest vendor dependency.

Table 8.1 — The six-pattern model of Claude × BIM integration (October 2026).

8.1 When Each Pattern Wins

- **Pattern A wins for:** single-call Revit add-ins shipped to end users.
- **Pattern B wins for:** multi-step agentic workflows that modify the live Revit model.
- **Pattern C wins for:** batch processing across many projects (requires Anthropic Enterprise + ZDR + DPA).
- **Pattern D wins for:** conversational queries and analytical workloads on a single .rvt or IFC file. Cross-platform, no install.
- **Pattern E wins for:** full agentic BIM workflows on IFC files (not native .rvt). Mutation, validation, clash, BCF, IDS authoring.
- **Pattern F wins for:** cross-tool AEC orchestration (Revit + Navisworks + ACC + Twinmotion + VR) without installing any of those tools. Requires accepting APS data egress.

8.2 The Strategic Shift from v1.0

v1.0 concluded Pattern B was dominant. v1.2 added Patterns D and E. v1.3 adds Pattern F and the three-pillar framing (rvt-rs + ifc-lite + BIMCraft). The revised conclusion: for analytical workloads (the majority of BIM management time), Patterns D and E are now viable and substantially better than Pattern B (lower latency, lower IP exposure, cross-platform, no Revit license cost). For multi-tool AEC orchestration across Revit + Navisworks + ACC + Twinmotion, Pattern F is the new option — trading data egress for capability breadth. Pattern B remains correct for live-model modification only.

Chapter 9. Enterprise Adoption Path

A phased path for an enterprise evaluating the three-pillar Rust BIM ecosystem.

9.1 Months 0-3: Pilot

Deploy rvt-rs, ifc-lite, and BIMCraft on an isolated workstation (any OS). Replicate the techdrawai blind test on a representative internal project: convert .rvt to IFC via rvt-rs OR via BIMCraft's convert command (2.3x faster), then run identical queries via ifc-lite's bim.* API and via the ifc-lite MCP server against Claude Desktop. Compare accuracy, latency, and per-task dollar cost against the existing IfcOpenShell-based workflow. Build a 20-task IFC eval corpus with known-correct answers.

9.2 Months 3-6: Internal Tooling

If the pilot meets accuracy thresholds (>85% on the 20-task corpus) and ROI thresholds (positive on at least one of CD production time, compliance review time, QA rework reduction), build an internal tooling wrapper around ifc-lite. The wrapper should expose a curated subset of the MCP tools to internal users, with audit logging, read-only defaults, and a plan/approve/execute/verify gate on every mutation. Negotiate Anthropic Enterprise + ZDR + EU residency (Anthropic Ireland) + BAA if health-adjacent projects are in scope. Evaluate BIMCraft for scenarios where a full BIM authoring tool is needed — but treat it as early-stage, with API churn expected.

9.3 Months 6-12: Production Decision

Decide between three paths: (1) scale build to production-grade internal tooling on ifc-lite, reserving Pattern B (Revit MCP) for live-model modification only; (2) adopt SWAPP or another commercial vendor as the primary path, with ifc-lite for internal analytical workloads; (3) evaluate Pattern F (ScanBIM-Labs or similar APS-cloud MCP) for cross-tool AEC orchestration if multi-tool workflows are the bottleneck. Decision criteria: accuracy on the 20-task corpus, total cost of ownership, IP/confidentiality risk, talent availability, and commercial vendor maturity.

9.4 The Talent Investment

Independent of the build-vs-buy decision, the firm should invest in at least one full-time equivalent in the emerging "AI BIM Engineer" role. Required skill profile: Revit API fluency in C#/.NET, pyRevit scripting, MCP server authoring (TypeScript or Python), IfcOpenShell (Python), and now Rust (cargo, wasm-pack, the rvt, ifc-lite, and BIMCraft crates). This role does not yet have an established job-title convention or salary benchmark. Early hires inside AI-BIM vendor startups (LTplus-AG, AECO MCPs, SWAPP, IPedrax/BIMCraft) are defining the role. An enterprise that builds this capability internally gains optionality on both the build and buy paths.

Chapter 10. 12-Month Forecast

Predictions for the Rust BIM ecosystem through late 2027.

10.1 High-Confidence Predictions

- **ifc-lite becomes the default for browser/WASM IFC viewing.** The WebGPU renderer, streaming parse, and 125-tool MCP surface are not matched by any competitor. Confidence: high.
- **rvt-rs survives as a single-author project through 2027, then is either acquired or forked.** BIMCraft's decision to build on rvt-rs rather than reimplement validates the foundation. If Griffin Long stops maintaining the project, a fork will emerge within 6 months.
- **BIMCraft becomes production-viable by Q3 2027.** The 2-week-old project already handles real 970-element buildings correctly. With 9-12 more months of maturation, it becomes a credible Revit alternative for specific niches (residential, small commercial). Will not displace Revit in enterprise by 2027.
- **IfcOpenShell and ifc-lite coexist through 2028.** IfcOpenShell retains the research and BlenderBIM ecosystem; ifc-lite captures the browser and high-performance server. The verified 100% agreement becomes a permanent feature of the relationship.

10.2 Medium-Confidence Predictions

- **A Rust Foundation working group for AEC forms by 2028.** The current fragmentation (openbimrs, cadmpeg, LTplus-AG, oxideav, holg, BIMCraft) is unsustainable. A working group with a shared test corpus, shared CI, and shared roadmap emerges.
- **The Giraffty publisher is identified by end of 2026.** Either the source code goes public (legitimate entrant) or it is linked to a commercial vendor (stealth launch).
- **Autodesk ships a Rust SDK for APS by 2029.** Currently raps is the only Rust client for APS. Autodesk notices ecosystem demand and ships an official SDK.

10.3 Low-Confidence Speculation

- **BIMCraft challenges Revit for residential architecture by 2028.** Not for enterprise, but for solo architects and small firms doing residential work where Revit is overkill.
- **Autodesk acquires LTplus-AG.** The ifc-lite MCP server is the most direct threat to Autodesk's control of the BIM data layer. An acquisition would neutralise that threat and give Autodesk a credible browser-first viewer.
- **The "AI BIM Engineer" role becomes a standard job title by 2028.** Defined by Revit API + pyRevit + MCP authoring + Rust (rvt-rs, ifc-lite, BIMCraft) + RAG. Salary benchmarks published by major firms.

10.4 The Bottom Line

The Rust BIM ecosystem in late 2026 has crossed from "interesting experiment" to "credible procurement option." The three-pillar foundation (rvt-rs for native .rvt access, ifc-lite for IFC processing with native MCP,

BIMCraft for full BIM authoring) is in place. The supporting cast (cadmpeg, openbimrs, oxideav-ifc, truck, raps, ScanBIM-Labs) fills in the gaps. The incumbents (IfcOpenShell, xBim, web-ifc, ThatOpen) are not going away, but they now have credible competition for the first time in a decade. For an enterprise, the question is when, and how much — not whether. The firms that pilot now will have a 3-5 year head start on the firms that do not.

Appendix A. Methodology

This book was researched across 21 rounds of verification on a Linux devpod in October 2026. Each round produced corrections, confirmations, or strategic findings documented in the companion errata-notes.md (980 lines). Nine independent comparison reports (in comparisons/) cover IfcOpenShell, xBim, web-ifc, Apache Tika, @phi-ag/rvt, openbim-ifc-cli, ThatOpen, BIMCraft, and the broader competitor landscape.

A.1 The 21 Verification Rounds

- Round 1-6: initial setup, install verification, real.rvt acquisition, smoke tests, subcommand sweep, online research on Windows/Desktop and BIM-domain edge cases.
- Round 7: proven eval exfiltration PoC (fetch + process.env) end-to-end.
- Round 8: verified --read-only mitigation hides 40/126 tools (mutation surface only, not eval/run which were never MCP tools).
- Round 9: final confirmation that eval/run are absent from tools/list (with or without --read-only).
- Round 10: full gap-list sweep — rvt-analyze/history/corpus/write, Python API 7-method sweep, anonymize inspection, bcf create zip structure.
- Round 11: CRITICAL — full sandbox escape sweep reveals arbitrary file read via dynamic import(), not just env leak. Sandbox provides no meaningful isolation.
- Round 12: second real file (Einhoven, Revit 2023) cross-version validation; config CLI quirk documented.
- Round 13: CRITICAL CORRECTION — 15ms parse figure was measured on wrong file (synthetic 1.8KB out.ifc, not real.ifc 4.8MB). Real figure: 90-160ms across 3 independent measurements.
- Round 13b: web-ifc vs ifc-lite comparison — refuted “beats web-ifc” speed claim; element counts identical; ifc-lite wins on agent-ergonomic API, not speed.
- Round 14: IfcOpenShell + xBim comparison — 100% agreement on real.ifc (stronger than claimed 99.9%), three-way cross-validation across rvt-rs/ifc-lite/xBim/IfcOpenShell/BIMCraft.
- Round 15: rvt-wasm publish status definitively resolved — does NOT exist on crates.io or npm.
- Round 16: phi-ag/rvt, Speckle, BCF tools comparison — phi-ag does metadata only (not element decode), Speckle is different category, openbimrs/bcf ships no CLI.
- Round 17: ThatOpen platform comparison — not a competitor, different niche; adoption 5-10x ifc-lite on core packages.
- Round 18: ifclite-geom Python wheel calling convention resolved (wants raw bytes, not path string). 417ms parse, 1,171 tessellated elements.
- Round 19: Apache Tika comparison — confirmed CLEANROOM.md overstates Tika’s .rvt support (generic OLE2 detection only, zero Revit-specific parsing).
- Round 20: BIMCraft verification — hands-on (built from source, tested info/convert/mcp), confirmed rvt-rs lineage, 2.3x faster than rvt-rs itself.

- Round 20b: openbim-ifc-cli comparison — 418K LOC Rust, real but young (3 stars, 2 months old), AGPL+commercial.
- Round 21: DEFINITIVE — ifc-lite is NOT primarily Rust (66% TS, 22% Rust per GitHub API). The “largest Rust BIM project” claim retired on two independent grounds.

A.2 Three Methodological Insights

A.2.1 The grep-partial-output False Negative

During Round 1, the researcher reported that ifc-lite ask did not exist as a CLI subcommand. The claim was based on running ifc-lite --help | tail -40 and ifc-lite --help | head -30 to save context. The ask line fell in the middle of the long help output that neither slice captured. The command existed the whole time. The fix was re-running ifc-lite --help | grep -i ask against the whole output. Generalisable rule: for any verification pass against a CLI tool, always capture the full --help output to a file and grep against the file. Never trust a pipe-through-head or pipe-through-tail for negative claims.

A.2.2 The Git LFS Pointer Stub Gotcha

When cloning the magnetar-io/revit-test-datasets repository via git clone --depth 1, the resulting .rvt files are ~130-byte Git LFS pointer stubs, not the actual binary model files. A shallow clone without Git LFS installed produces stub files that look correct (right filename, right path) but fail to parse. The workaround: download the binary directly from GitHub’s media CDN via curl https://media.githubusercontent.com/media///main/. This bypasses Git LFS entirely. Generalisable rule: when verifying a tool against a test dataset hosted on GitHub, check whether the repository uses Git LFS (git lfs ls-files). If it does and Git LFS is not installed, install Git LFS or download files directly from the media CDN.

A.2.3 The Wrong-File Measurement Error

The 15ms parse time reported in v1.2 was measured on out.ifc (1.8KB, 24 entities, synthetic scaffold file) but quoted as if it applied to real.ifc (4.8MB, 64,747 entities, 970 real building elements). The figure got carried forward through multiple rounds without re-verification because it sounded right and matched the expectation of fast. Three independent agent sessions (web-ifc comparison, IfcOpenShell comparison, direct re-test) caught it. Generalisable rule: any performance figure measured on a test input should be re-measured on the production input before being quoted. The cost of re-measurement is negligible; the cost of a wrong performance claim propagated into a published book is significant.

Appendix B. Sources & Companion Documents

B.1 Primary GitHub Repositories

- DrunkOnJava/rvt-rs — https://github.com/DrunkOnJava/rvt-rs (raw README, CLEANROOM.md, THREAT_MODEL.md, ROADMAP.md, CHANGELOG.md, AGENTS.md)
- LTplus-AG/ifc-lite — https://github.com/LTplus-AG/ifc-lite (raw README, AGENTS.md, CLAUDE.md, .claude/skills/use-ifclite/SKILL.md, RELEASE.md, SECURITY.md, LICENSE_HEADER.md)
- IPedrax/BIMCraft — https://github.com/IPedrax/BIMCraft (verified 743 stars, Apache-2.0, Rust, rvt-rs port)
- cadmpeg/cadmpeg — https://github.com/cadmpeg/cadmpeg (12 native CAD format readers)
- OxideAV/oxideav-ifc — https://github.com/OxideAV/oxideav-ifc (pure-Rust IFC reader, Karpelès Lab)
- openbimrs/ifc — https://github.com/openbimrs (full buildingSMART stack, AGPL+commercial)
- phi-ag/rvt — https://github.com/phi-ag/rvt (TypeScript .rvt container reader)
- MetabuildDev/ifc_rs — https://github.com/MetabuildDev/ifc_rs (alpha-stage Rust IFC, last updated Dec 2024)
- ScanBIM-Labs/scanbim-mcp — https://github.com/ScanBIM-Labs/scanbim-mcp (Pattern F: APS-mediated cloud MCP)
- apache/tika — https://github.com/apache/tika (verified: no Revit-specific parser, generic OLE2 only)
- giraffty org — https://github.com/giraffty (0 public repos as of 2026-10-30)
- magnetar-io/revit-test-datasets — https://github.com/magnetar-io/revit-test-datasets (Core Interior + Einhoven test models, MIT)

B.2 Package Registries

- crates.io API — https://crates.io/api/v1/crates/{crate} (queried for: rvt, ifc-lite-core, ifc-lite-geometry, ifc-lite-clash, ifc-lite-wasm, ifc-lite-processing, oxideav-ifc, openbim-ifc, ifc_rs, cadmpeg-codec-f3d, raps, rvt-cli, eulumdat-*)
- npm registry — https://registry.npmjs.org/-/v1/search?text=%40ifc-lite (50+ @ifc-lite packages); https://api.npmjs.org/downloads/point/last-month/{package} (per-package download counts)
- PyPI — https://pypi.org/pypi/rvt/json (12 releases, author email Griffin Long); https://pypi.org/pypi/ifclite-geom/json (native ifc-lite geometry-data export)
- docs.rs — https://docs.rs/rvt/latest/rvt/ (17 public modules); https://docs.rs/giraffty-connection/latest/giraffty_connection/; https://docs.rs/giraffty-transport/latest/giraffty_transport/
- GitHub languages API — https://api.github.com/repos/LTplus-AG/ifc-lite/languages (TypeScript 66%, Rust 22%, JavaScript, C#, etc.)

B.3 Companion Documents

- The Rusty Revit Book v1.0 (October 2026) — original 43-page book. Superseded by v1.3 in the sections noted in Table 0.1.
- The Rusty Revit Book v1.2 Verified Edition (October 2026) — 17-page errata + operational guide. Superseded by v1.3.
- Claude x Revit: Enterprise Strategy and Risk Assessment (October 2026) — 34-page deep-dive enterprise report. Still valid; Pattern D/E/F additions in v1.3 supplement (not replace) the enterprise report.
- Claude x Revit Friends House Session Playbook (October 2026) — 19-page practical testing playbook. Section 4 of v1.3 (Operational Guide) supersedes Scenario 7 of the playbook.
- lafinak/revit verification repo — https://github.com/lafinak/revit (980-line errata-notes.md, 9 comparison reports, verified test artifacts). The complete verification dossier underlying this book.

B.4 What v1.4 Will Add

- Full bim.* API surface from ifc-lite schema output (currently verified for 10+ queries, full catalogue pending)
- Complete ifc-lite ask recipe catalogue (verified recipes: count-walls, window-wall-ratio, exterior-walls; full list pending)
- IDS authoring walkthrough using ifc-lite ids audit|lint|fmt subcommands
- BCF topic creation and viewpoint authoring with ifc-lite bcf create|list|add-comment
- Full 126-tool MCP catalogue (currently verified by count; per-tool documentation pending)
- Extended sandbox escape sweep beyond import()/process.env/fetch/WebSocket/Function/eval (currently 12 vectors tested, more pending)
- BIMCraft source code audit — what was ported from rvt-rs vs original; full feature surface coverage
- ScanBIM-Labs live test (requires APS credentials)
- Real-project findings from the friend's house session (different Revit version, different element mix, possibly linked models)

v1.3 was produced after 21 rounds of hands-on verification on a Linux devpod, cross-validated by 9 independent comparison reports. It supersedes v1.0 and v1.2 in the sections noted; the rest of v1.0 remains the reference. v1.4 will follow after the friend's house session and will incorporate findings from the friend's real project IFC export. The ecosystem is moving fast; verify against primary sources before making procurement decisions.